Practice Midterm 2
Introduction to Algorithms
601.433/633

Name:

Ethics Statement

I agree to complete this exam without unauthorized assistance from any person, materials, or device.

Signature: Date:

1 True/False (15 points)

Indicate whether each statement is true or false. You **do not** need to give a proof, nor do you need to give counter-examples.

(a) If I_1 and I_2 are independent sets in a matroid with $|I_1| > |I_2|$, then $I_2 \cup \{e\}$ is independent for all $e \in I_1 \setminus I_2$.
true **false**

(b) If all edge weights are distinct in a graph, then there is only one MST.
true false

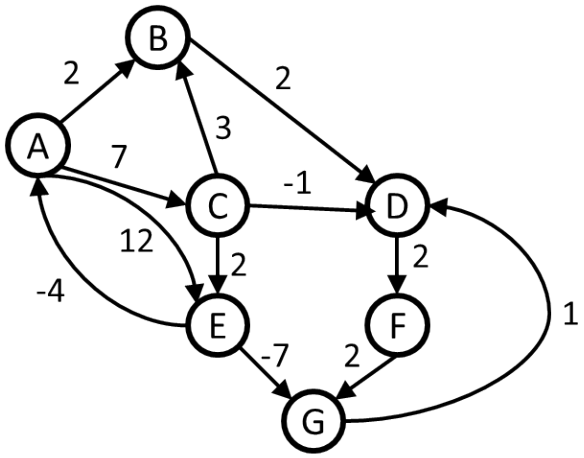
(c) In a bottom-up dynamic programming algorithm with a 2-dimensional table with n rows and m columns, we can always fill in the entries via two nested for loops of the form: for($i = 1$ to n) for($j = 1$ to m) $M[i, j] = \ldots$
true **false**

(d) Bellman-Ford gives correct distances even when there are negative edge weights, as long as there are no negative-weight cycles.
true false

(e) The node with smallest finishing time in a call to DFS(G) is in a source strongly connected component.
true **false**

2 Shortest Paths (30 points)

Consider the following graph.



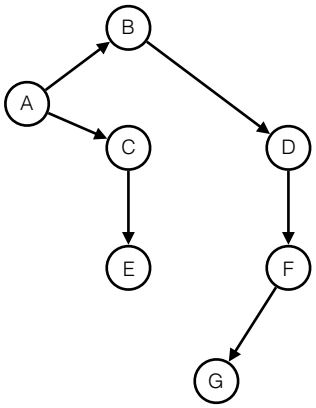
Even though the graph has negative weight edges, let's consider what happens if we run Dijkstra's algorithm.

(a) (15 points) Let T be the shortest-path tree constructed by Dijkstra's algorithm on G (starting from A). List the vertices of G in the order in which they were added to T.

Solution: A B D F C G E

(b) (10 points) Draw the tree T computed by Dijkstra's algorithm on G (starting from A).

Solution:



(c) (5 points) Dijkstra's algorithm found the wrong path to some nodes. For one of these nodes, write down the actual shortest path.

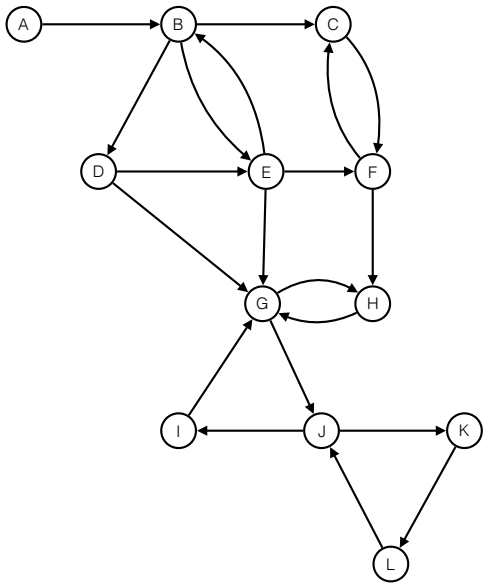
Solution: G: A C E G

D: A C E G D

F: A C E G D F

3 Strongly Connected Components (15 points)

List the strongly connected components of the following graph, and give a topological sort of the graph of the SCCs (i.e., the graph obtained by creating a vertex for each SCC and adding an edge from u to v if there is an edge in the given graph from the SCC that u represents to the SCC that v represents)



Solution: We have the following four SCCs:

$$C_1 = \{A\}$$
$$C_2 = \{B, D, E\}$$
$$C_3 = \{C, F\}$$
$$C_4 = \{G, H, I, J, K, L\}$$

A topological sort of the SCC graph is C_1, C_2, C_3, C_4.

4 Kruskal's Algorithm (15 points)

Assume you have an implementation of the Union-Find data structure that has an amortized time of $f(n)$ for Union operations and amortized time of $g(n)$ for Find operations. If you use this data structure to implement Kruskal's algorithm on a connected graph with n nodes and m edges, what will be the running time of Kruskal's? Explain your answer.

Solution: When we run Kruskal's algorithm we need to sort the edges, and then for every edge we do a Find to determine if the endpoints are in the same component. The edges which cause a Union are the ones that form the MST found by the algorithm, so there are at most $n-1$ Union operations. Thus the total running time would be $O(m \log m + m \cdot g(n) + n \cdot f(n)) = O(m \log n + m \cdot g(n) + n \cdot f(n))$.

5 Dynamic Programming (25 points)

Let $G = (V, E)$ be a directed graph with vertices $v_1, v_2, \ldots, v_n$. Recall that $|E| = m$. We say that G is an *ordered graph* if it has the following two properties:

1. Every edge goes from a vertex with a lower index to a vertex with a higher index. That is, every edge has the form (v_i, v_j) with $i < j$.

2. Every vertex other than v_n has at least one outgoing edge.

The length of a path is the number of edges on it. In this problem you will design an efficient algorithm to find the length of the *longest* path which starts at v_1 and ends at v_n. Note that by the two properties of an ordered graph, every node has at least one path to v_n.

(a) (10 points) Let $S(i)$ denote the length of the longest path that starts at v_i and ends at v_n. Write a (recursive) formula for $S(i)$, which in the next part will form the basis of a dynamic programming algorithm. You do not need a formal proof of correctness, but should justify your answer.

Solution: First, note that $S(i)$ is well-defined for all i because (by the second property of ordered graphs) every node has at least one path to v_n. Note that the longest path from v_i to v_n must have some first hop using edge (v_i, v_j) where $j > i$. Thus we have the formula

$$S(i) = \begin{cases} 0 & \text{if } i = n, \\ 1 + \max_{j:(v_i,v_j)\in E}\{S(j)\} & \text{otherwise} \end{cases}$$

(b) (15 points) Design an $O(m)$-time algorithm to find the longest path which starts at v_1 and ends at v_n (using your solution from part (a)). Argue correctness and running time (again, you do not need a fully formal proof, but should provide justification).

Solution: Consider the following algorithm (which is just a bottom-up dynamic programming algorithm for the formula from part (a)), which returns the *length* of the longest path but not the longest path itself.

$M[n] = 0$
for $(i = n - 1$ downto $1)$ {
$M[i] = 1 + \max_{j:(v_i,v_j)\in E} M[j]$
}
return $M[1]$

For correctness, we know by construction that $M[n] = S(n)$, and then a simple induction (together with part (a)) implies that $M[i] = S(i)$ for all i. Hence $M[1] = S(1)$, which by definition is the value we are looking for.

For the running time, let d_i be the outdegree of v_i. Note that the time it takes to calculate $M[i]$ is $O(d_i)$. Hence the total running time is $\sum_{i=1}^{n} O(d_i) = O(m)$.

Once the above algorithm correctly fills out M, we can use the standard backtracking through the dynamic programming table to compute the longest path itself. Slightly more formally (not necessary that students actually write this out), we can use the following algorithm to give the actual path.

$i = 1$
Path $P = (i)$
while $(i \neq n)$ {
iterate through the edges out of v_i until we find the edge (i, j) where $M[i] == M[j] + 1$
Add j to the path.
set $i \leftarrow j$
}

The running time of this is clearly at most $O(m)$, and since the previous algorithm fills out M correctly we know that it will correctly find the longest path.